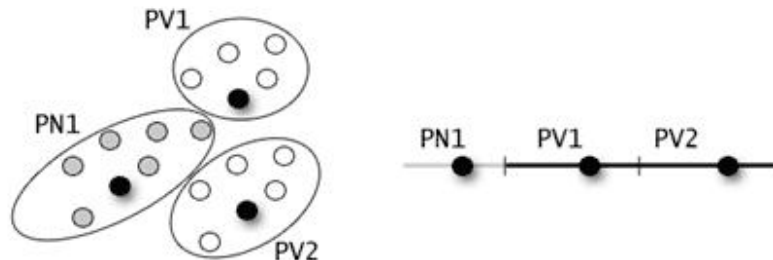


Receta: testeo de dominio mediante particiones equivalentes

1. Cuando utilizar esta receta

Esta receta resulta útil para testear si componentes o funciones individuales **calculan correctamente valores de salida a partir de sus entradas**, siempre que las entradas se puedan dividir en **particiones con características similares**.



Cabe señalar que estas condiciones son muy generales y, por tanto, esta receta puede usarse en multitud de casos; por ejemplo, a nivel de código fuente, para **testear subprogramas** (funciones, métodos); en el testeo de **interfaces de usuario con varios campos**, en las que cada campo es una variable de entrada con su propio dominio; etc.

Desarrollemos un ejemplo a medida que definimos la receta para comprender mejor cada uno de sus pasos:

EJEMPLO: A una discoteca pueden acceder, pagando entrada, mayores de edad (18 años o más), siendo necesario que proporcionen su número de DNI o de pasaporte. Si proporcionan un número válido de tarjeta VIP no se comprueba su edad ni pagan entrada.

El acceso a la discoteca incluye una función que, a partir de la edad del cliente y del tipo de documento que presenta, calcula si debe permitir su acceso gratis o pagando entrada, o bien denegar su acceso:

```
acceso_disco (edad, núm.documento) = {gratis|pagando|denegar}
```

¿Qué casos de test podemos elegir para comprobar que no existen errores en su implementación?

Se trata de una situación a la que esta receta (testeo de dominio) se adapta perfectamente: comprobar valores de salida en función de las entradas, que además se pueden dividir en particiones con funcionamiento equivalente (mayor edad/menor edad; documento aceptado/no aceptado; etc.).

2. El modelo del software: los dominios divididos en particiones equivalentes

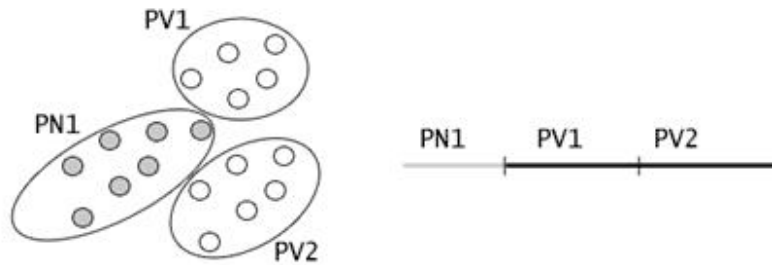
El modelo del SBT (Software Bajo Testeo) en esta receta consiste en una colección de **particiones equivalentes para cada entrada** al sistema; una partición equivalente es un subdominio de una entrada cuyos miembros se procesan de forma equivalente.

Cómo construir el modelo.

Los pasos a seguir para obtener el modelo del software son:

1. **Identificar todas las funciones** (comportamiento, procesamiento, componentes, métodos, etc.) del SBT.
2. Para cada función, **determinar sus entradas y salidas**.
3. **Examinar los dominios de cada entrada**, es decir, los tipos de valores que estas entradas pueden tener.
4. **Dividir el dominio de cada entrada en particiones equivalentes**, que serán subdominios de la entrada cuyos miembros son procesados del mismo modo por el sistema. Cada partición puede ser **válida** (valores

l d l f ó álid l l d d f d



Los siguientes **criterios** pueden resultar útiles para definir las particiones equivalentes, teniendo en cuenta que deben servir como guía y no como reglas absolutas:

1. Si una condición de entrada especifica una condición que debe cumplirse (“debe ser un número de documento aceptado”), identificar una partición válida con los valores que cumplen la condición ({documentos aceptados}) y otra partición no válida con los que no la cumplen ({documentos no aceptados}).
2. Si una condición de entrada especifica un conjunto de valores (ej.: “números DNI, pasaporte y tarjetas VIP”), crear una partición válida para cada subconjunto de valores que sean tratados igual (en este caso, dos particiones: {DNIs y pasaportes} y {VIPs}), y una partición no válida con los valores no incluidos en ese conjunto ({documentos no aceptados}).
3. Si una condición de entrada incluye un rango de valores (ej.: “la edad es mayor o igual que 0”), crear una partición válida para los valores dentro del rango ($\text{edad} \geq 0$) y particiones no válidas para los valores fuera del rango ($\text{edad} < 0$).
4. Si se sospecha que los elementos dentro de una partición no serán tratados de modo idéntico, dividir la partición en otras más pequeñas (ej.: si se trata de modo diferente a mayores y menores de edad, dividimos la partición $\text{edad} \geq 0$ en dos: $0 \leq \text{edad} \leq 17$ y $\text{edad} \geq 18$).

En la función del ejemplo, las entradas son `edad` y `documento`; la salida es la de la función `acceso_disco`.

Podemos definir estas particiones:

	Dominio	Particiones válidas	Particiones no válidas
edad (entrada)	números enteros	PV1: $0 \leq \text{edad} \leq 17$ PV2: $\text{edad} \geq 18$	PN1: $\text{edad} < 0$
documento (entrada)	números de documento	PV1: {dnis y pasaportes} PV2: {vips}	PN1: {documentos no aceptados}

Nótese que no es necesario incluir la partición {documentos aceptados} ya que ésta de hecho está cubierta por las particiones PV1 y PV2

3. verificación del modelo

Para verificar nuestro modelo, habrá que revisar si dentro de cada partición sus elementos son efectivamente equivalentes; es decir, cumplen que:

- Tienen las mismas características, escenario y/o procesamiento.
- Si uno de ellos produce un fallo, los otros (probablemente) también.
- Si uno de ellos no revela ningún fallo, los otros (probablemente) tampoco.

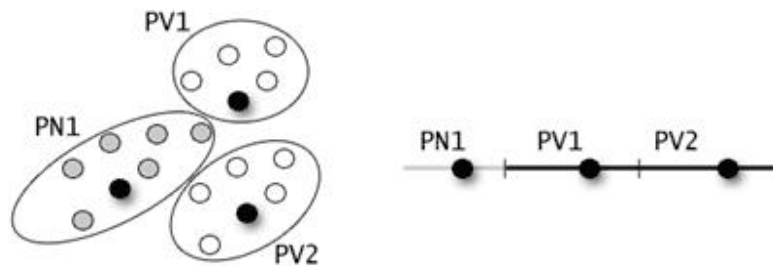
Revisando las particiones del ejemplo:

- Para la entrada `edad`, todos los valores de PV1 (menor de edad) deben ser tratados del mismo modo; lo

similar.

4. Criterio para elegir casos de test: un representante de cada partición

En primer lugar es necesario seleccionar valores concretos con los que se generarán los casos de test. Esto se realiza eligiendo un valor representante por cada partición definida en el paso anterior.



El testeador utilizará sus conocimientos del sistema y su experiencia para la elección de los valores concretos. En todo caso, como anteriormente hemos verificado el modelo y sabemos que cumple con las condiciones mencionadas anteriormente, podemos suponer que **cualquier valor de la partición será representativo de las características y funciones del sistema**. Por ejemplo, si hemos particionado el dominio de una entrada en tres particiones, entonces elegimos simplemente un valor de cada partición (tres valores en total).

En nuestro ejemplo, elegimos un representante cualquiera para cada partición:

	Particiones	Representante
edad (entrada)	PV1: $0 \leq \text{edad} \leq 17$	10
	PV2: $\text{edad} \geq 18$	25
	PN1: $\text{edad} < 0$	-5
documento (entrada)	PV1: {dnis y pasaportes}	PASAPORTE_VÁLIDO
	PV2: {vips}	VIP_VÁLIDO
	PN1: {documentos no acept.}	0

PASAPORTE_VÁLIDO es cualquier número de pasaporte válido que se sabe que es aceptado por el sistema (igualmente podríamos haber elegido un número de DNI); VIP_VÁLIDO es cualquier número de tarjeta VIP que se sabe que es aceptada por el sistema.

5. Generar casos de test

En este punto, los casos de test (es decir, los conjuntos de entradas y salidas esperadas que deberán comprobarse) se generan combinando los representantes seleccionados en el paso anterior para cada entrada. Para combinarlas hay que tener en cuenta estas reglas:

Particiones válidas

Para generar casos de test, cada representante de las particiones válidas se combina con los representantes del resto de particiones de todos los modos posibles. Calculando el número de casos de test generados por particiones válidas tenemos que:

Núm. casos de test (particiones válidas) = multiplicación del número de particiones válidas de cada entrada

Recopilando los valores representantes seleccionados anteriormente:

	Valores válidos	Valores no válidos
edad (entrada)	10, 25	-5

edad (entrada)	10	10	25	25
documento (entrada)	PASAPORTE_VÁLIDO	VIP_VÁLIDO	PASAPORTE_VÁLIDO	VIP_VÁLIDO
acceso_disco (SALIDA ESPERADA)	denegar	gratis	pagando	gratis

Particiones no válidas

A pesar de que la función no contemple los valores de estas particiones, entendemos que deben incluirse en los casos de test para comprobar que el sistema responde de modo **robusto** (por ejemplo, devolviendo un mensaje de error en vez de quedar bloqueado) si se produce una entrada no válida; la experiencia muestra que muchos errores reales están motivados por llamadas a funciones con valores que, en teoría, “no podían suceder”.

En este caso, consideramos suficiente generar **un caso de test para cada partición no válida, tomando para el resto de entradas cualquier representante válido**. No combinamos particiones no válidas entre sí, ya que los errores producidos proporcionarían pocas pistas sobre la entrada que los causa (los valores no válidos en una entrada podrían enmascarar errores producidos por valores no válidos de otras entradas).

De ese modo, el número de casos de test que incluyen particiones no válidas es:

Núm. casos de test (particiones no válidas) = suma del número de particiones no válidas de cada entrada.

De igual modo utilizando las particiones no válidas del ejemplo, generamos estos 2 casos de test:

	<u>casos de test</u>	
	C5	C6
edad (entrada)	-5	25
documento (entrada)	PASAPORTE_VÁLIDO	0
acceso_disco (SALIDA ESPERADA)	?	?
	(no definida)	(no definida)

Concluyendo con el ejemplo, hemos obtenido de este modo 4 (2x2) casos de test con particiones válidas, y 2 (1+1) con particiones no válidas. Así pues, el testeo de la función usando esta receta consistiría en **comprobar que:**

```
acceso_disco(10, PASAPORTE_VÁLIDO) = denegar //caso de test C1
acceso_disco(10, VIP_VÁLIDO) = gratis //caso de test C2
acceso_disco(25, PASAPORTE_VÁLIDO) = pagando //caso de test C3
acceso_disco(25, VIP_VÁLIDO) = gratis //caso de test C4
```

Y en **comprobar que** el sistema responde de modo adecuado a estas entradas no válidas (por ejemplo, devolviendo un error pero permitiendo que se siga utilizando sin quedar “colgado”):

```
acceso_disco(-5, PASAPORTE_VÁLIDO) //caso de test C5
acceso_disco(25, 0) //caso de test C6
```

6. Herramientas; referencias; ejemplos

Algunos de los pasos y técnicas utilizados en esta receta han sido adaptados a partir de otros encontrados en diversos libros y artículos bajo diferentes nombres: testeo de dominio; testeo de particiones (categorías) equivalentes; testeo de combinación de datos; testeo de interfaces de programa; testeo sintáctico

Un ejercicio

Como ejercicio, dejamos al lector este ejemplo para que compruebe que se adapta a esta receta, y que pueda seguir sus pasos para la generación de casos de test.

Imaginemos que tenemos que testear un método `coste_administracion(Money m)` que acepta como parámetro un objeto `Money m` definido así:

```
class Money {
    private int fAmount;
    private String fCurrency;

    public Money(int amount, String currency) {
        fAmount= amount;
        fCurrency= currency;
    }

    public int amount() {
        return fAmount;
    }

    public String currency() {
        return fCurrency;
    }
}
```

El método `coste_administracion` funciona correctamente con estos supuestos:

- `m.amount() ≥ 0`
 - `m.currency() in {EUR, USD, GBP, YEN}`
- calculando los costes de administración de este modo:
- Si el cliente quiere pagar en USD, GBP o YEN, el tiene que pagar 10% de costes de administración
 - Si el cliente paga en EUR no hay costes de administración

¿Qué casos de test podríamos utilizar para validar `coste_administracion`?

[Cerrar ventana](#)